HW01

MIT 6.792 Reinforcement Learning

Gatlen Culp (gculp@mit.edu) · Monday, 2025 Oct 27
Collaborators: Claude 4.5 Sonnet (Questions, not Solving) · Resources Used: None

Notes

test

1 Actor-Critic Methods

For convenience, we define the discounted total reward of a trajectory τ as:

$$R(\tau) := \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t), \tag{1}$$

where s_t, a_t are the state-action pairs in τ. Observe that:

$$V_\mu(\pi_\theta) = \mathbb{E}_{\tau \sim P_\mu^{\pi_\theta}}[R(\tau)], \tag{2}$$

For some initial state distribution, $\gamma = 1$ and a finite-horizon ending at T, we have seen in class that

$$\nabla_\theta V_\mu(\pi_\theta) = \mathbb{E}_{\tau \sim P_\mu^{\pi_\theta}} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]. \tag{3}$$

In this problem, we will show that a similar expression can be derived for the gradient of the value function in terms of action-value functions Q. This introduction of the critic Q lends itself to actor-critic methods, both basic and advanced. Assume that policies are stochastic.

1.1 Part 1

Derive the policy gradient for the discounted infinite horizon case (note that the derivation in lecture is for the finite horizon one). In other words, prove that for a discount factor $\gamma < 1$, the policy gradient is

$$\nabla_\theta V_\mu(\pi_\theta) = \mathbb{E}_{\tau \sim P_\mu^{\pi_\theta}} \left[R(\tau) \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right] \tag{4}$$

For simplicity, we will alias some of the variables:

$$\pi = \pi_\theta, \quad P = P_\mu^{\pi_\theta}, \quad \nabla = \nabla_\theta, \quad V_\mu(\pi_\theta) = V(\theta) \tag{5}$$

We can use the same likelihood rescaling trick from lecture. From above:

$$V(\theta) = \mathbb{E}_{\tau \sim P}[R(\tau)] \tag{6}$$

Where $R(\tau)$ completely handles the γ term and infinite sum. We are interested in the policy gradient:

$$\nabla_\theta V(\theta) = \nabla_\Delta V(\theta + \Delta)|_{\Delta=0} \tag{7}$$

Firstly, we can get $V(\theta + \Delta)$ by using likelihood rescaling – the discounted total trajectory is scaled according to the relative likelihood of generating that trajectory given the change in parameters:

$$V(\theta + \Delta) = \mathbb{E}_{\tau \sim P} \left[R(\tau) \frac{\prod_t \pi_{\theta+\Delta}(a_t \mid s_t)}{\prod_t \pi_\theta(a_t \mid s_t)} \right] \tag{8}$$

To find $\nabla_\Delta V(\theta + \Delta)|_{\Delta=0}$, first note that the expectation is independent of Δ, so it can be moved in immediately. Δ is also unrelated to $R(\tau)$. So:

$$\nabla_\Delta V(\theta + \Delta)|_{\Delta=0} = \mathbb{E}_{\tau \sim P} \left[\frac{R(\tau)}{\prod_t \pi_\theta(a_t \mid s_t)} \nabla_\Delta \left(\prod_t \pi_{\theta+\Delta}(a_t \mid s_t) \right) \right] \Bigg|_{\Delta=0} \tag{9}$$

Focusing just on term with the gradient:

$$\nabla_\Delta \left(\prod_t \pi_{\theta+\Delta}(a_t \mid s_t) \right) = \sum_{t=0}^{\infty} \left[\nabla_\Delta \pi_{\theta+\Delta}(a_t \mid s_t) \prod_{t' \neq t} (\pi_{\theta+\Delta}(a_{t'} \mid s_{t'})) \right] \tag{10}$$

Substiting:

$$\nabla_\Delta V(\theta + \Delta)|_{\Delta=0} = \mathbb{E}_{\tau \sim P} \left[\frac{R(\tau)}{\prod_t \pi_\theta(a_t \mid s_t)} \sum_{t=0}^{\infty} \left[\nabla_\Delta \pi_{\theta+\Delta}(a_t \mid s_t) \prod_{t' \neq t} (\pi_{\theta+\Delta}(a_{t'} \mid s_{t'})) \right] \right] \Bigg|_{\Delta=0} \tag{11}$$

$$\tag{12}$$

$$= \mathbb{E}_{\tau \sim P} \left[\frac{R(\tau)}{\prod_t \pi_\theta(a_t \mid s_t)} \sum_{t=0}^{\infty} \left[(\nabla_\theta \pi_\theta(a_t \mid s_t)) \prod_{t' \neq t} (\pi_\theta(a_{t'} \mid s_{t'})) \right] \right] \tag{13}$$

$$\tag{14}$$

Make prod independent of t and factor it out $\tag{15}$

$$= \mathbb{E}_{\tau \sim P}\left[\frac{R(\tau)}{\prod_t \pi_\theta(a_t \mid s_t)} \sum_{t=0}^{\infty}\left[(\nabla_\theta \pi_\theta(a_t \mid s_t))\frac{\prod_{t'}(\pi_\theta(a_{t'} \mid s_{t'}))}{\pi_\theta(a_t \mid s_t)}\right]\right] \tag{16}$$

$$= \mathbb{E}_{\tau \sim P}\left[\frac{R(\tau)}{\prod_t \pi_\theta(a_t \mid s_t)}\left(\prod_t \pi_\theta(a_t \mid s_t)\right) \sum_{t=0}^{\infty}\left[\frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}\right]\right] \tag{17}$$

$$= \mathbb{E}_{\tau \sim P}\left[\frac{R(\tau)}{\prod_t \pi_\theta(a_t \mid s_t)}\left(\prod_t \pi_\theta(a_t \mid s_t)\right) \sum_{t=0}^{\infty}\left[\frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}\right]\right] \tag{18}$$

$$= \mathbb{E}_{\tau \sim P}\left[R(\tau) \sum_{t=0}^{\infty}\left[\frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}\right]\right] \tag{19}$$

Since $\nabla_\theta \log \pi_\theta = \frac{\nabla_\theta \pi_\theta}{\pi_\theta}$:

$$\nabla_\theta V_\mu(\pi_\theta) = \mathbb{E}_{\tau \sim P}\left[R(\tau) \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] \tag{20}$$

■

Note: This didn't actually feel easier than the common proof, I'm not sure how the jump from Likelihood scaling to chain rule worked.

1.2 Part 2

Derive the policy gradient for the actor-critic for the discounted infinite horizon case, that is, show that policy gradients can be expressed as

$$\nabla_\theta V_\mu(\pi_\theta) = \mathbb{E}_{\tau \sim P_\mu^{\pi_\theta}} \left[\sum_{t=0}^{\infty} \gamma^t Q^{\pi_\theta}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right] \tag{21}$$

and

$$\nabla_\theta V_\mu(\pi_\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_\mu^{\pi_\theta}} \mathbb{E}_{a \sim \pi_\theta(\cdot \mid s)} [Q^{\pi_\theta}(s, a) \nabla_\theta \log \pi_\theta(a \mid s)], \tag{22}$$

where the discounted future state distribution $d_\mu^{\pi_\theta}$ is defined as

$$d_\mu^{\pi_\theta}(s) = (1-\gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid \pi_\theta, \mu) \tag{23}$$

Note: d should have an overline

For simplicity, we will alias some of the variables:

$$\pi = \pi_\theta, \quad P = P_\mu^{\pi_\theta}, \quad \nabla = \nabla_\theta, \quad V_\mu(\pi_\theta) = V(\theta), \quad Q^{\pi_\theta} = Q, \quad d_\mu^{\pi_\theta} = d \tag{24}$$

Proving Equation 21

Let's start from the derivation in part 1 and expand the product of sums:

$$\nabla_\theta V_\mu(\pi_\theta) = \mathbb{E}_{\tau \sim P} \left[R(\tau) \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right] \tag{25}$$

$$= \mathbb{E}_{\tau \sim P} \left[\left(\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right) \left(\sum_{t'=0}^{\infty} \nabla_\theta \log \pi_\theta(a_{t'} \mid s_{t'}) \right) \right] \tag{26}$$

$$= \mathbb{E}_{\tau \sim P} \left[\sum_{t=0}^{\infty} \sum_{t'=0}^{\infty} \gamma^t r(s_t, a_t) \nabla_\theta \log \pi_\theta(a_{t'} \mid s_{t'}) \right] \tag{27}$$

$$= \mathbb{E}_{\tau \sim P} \left[\sum_{t'=0}^{\infty} \nabla_\theta \log \pi_\theta(a_{t'} \mid s_{t'}) \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \tag{28}$$

$$= \mathbb{E}_{\tau \sim P} \left[\sum_{t'=0}^{\infty} \nabla_\theta \log \pi_\theta(a_{t'} \mid s_{t'}) \left(\sum_{t=0}^{t'-1} \gamma^t r(s_t, a_t) + \sum_{t=t'}^{\infty} \gamma^t r(s_t, a_t) \right) \right] \tag{29}$$

The first term represents rewards before time t', which are independent of action $a_{t'}$. By the causality principle, this term contributes zero to the gradient. Therefore:

$$= \mathbb{E}_{\tau \sim P} \left[\sum_{t'=0}^{\infty} \nabla_\theta \log \pi_\theta(a_{t'} \mid s_{t'}) \sum_{t=t'}^{\infty} \gamma^t r(s_t, a_t) \right] \tag{30}$$

Reindex the inner sum with $k = t - t'$, so $t = t' + k$:

$$= \mathbb{E}_{\tau \sim P}\left[\sum_{t'=0}^{\infty} \nabla_\theta \log \pi_\theta(a_{t'} \mid s_{t'}) \sum_{k=0}^{\infty} \gamma^{t'+k} r(s_{t'+k}, a_{t'+k})\right] \tag{31}$$

$$= \mathbb{E}_{\tau \sim P}\left[\sum_{t'=0}^{\infty} \gamma^{t'} \nabla_\theta \log \pi_\theta(a_{t'} \mid s_{t'}) \sum_{k=0}^{\infty} \gamma^{k} r(s_{t'+k}, a_{t'+k})\right] \tag{32}$$

Note that the inner sum is the Q-function:

$$Q^\pi(s_{t'}, a_{t'}) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r(s_{t'+k}, a_{t'+k}) \mid s_{t'}, a_{t'}\right] \tag{33}$$

Therefore (renaming t' back to t):

$$\nabla V(\pi) = \mathbb{E}_{\tau \sim P}\left[\sum_{t=0}^{\infty} \gamma^t Q(s_t, a_t) \nabla \log \pi(a_t \mid s_t)\right] \tag{34}$$

This proves Equation 21. ■

Proving Equation 22

Where the discounted future state is:

$$d(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^t \mathbb{P}(s_t = s \mid \pi, \mu) \tag{35}$$

We will work backward from Equation 22 to get Equation 21 which we have shown above to be an equivalent formulation of the policy gradient.

$$\boldsymbol{\nabla} V(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d}\mathbb{E}_{a\sim\pi(\cdot \mid s)}[Q(s,a)\boldsymbol{\nabla}\log\pi(a \mid s)] \tag{36}$$

$$= \frac{1}{1-\gamma}\left[\int_s (1-\gamma)\sum_{t=0}^{\infty}\gamma^t \mathbb{P}(s_t = s \mid \pi, \mu)\mathbb{E}_{a\sim\pi(\cdot \mid s)}[Q(s,a)\boldsymbol{\nabla}\log\pi(a \mid s)]\,\mathrm{d}s\right] \tag{37}$$

$$= \int_s \sum_{t=0}^{\infty}\gamma^t \mathbb{P}(s_t = s \mid \pi, \mu)\left(\int_a \pi(a \mid s)Q(s,a)\boldsymbol{\nabla}\log\pi(a \mid s)\,\mathrm{d}a\right)\mathrm{d}s \tag{38}$$

$$= \int_s \int_a \sum_{t=0}^{\infty}\gamma^t \mathbb{P}(s_t = s \mid \pi, \mu)\pi(a \mid s)Q(s,a)\boldsymbol{\nabla}\log\pi(a \mid s)\,\mathrm{d}a\,\mathrm{d}s \tag{39}$$

$$= \int_s \int_a \sum_{t=0}^{\infty}\gamma^t \mathbb{P}(s_t = s, a_t = a \mid \pi, \mu)Q(s,a)\boldsymbol{\nabla}\log\pi(a \mid s)\,\mathrm{d}a\,\mathrm{d}s \tag{40}$$

$$= \sum_{t=0}^{\infty}\gamma^t \int_s \int_a \mathbb{P}(s_t = s, a_t = a \mid \pi, \mu)Q(s,a)\boldsymbol{\nabla}\log\pi(a \mid s)\,\mathrm{d}a\,\mathrm{d}s \tag{41}$$

$$= \sum_{t=0}^{\infty}\gamma^t \int_\tau \mathbb{P}(\tau \mid \pi, \mu)Q(s_t,a_t)\boldsymbol{\nabla}\log\pi(a_t \mid s_t)\,\mathrm{d}\tau \tag{42}$$

$$= \sum_{t=0}^{\infty}\gamma^t \int_\tau \mathbb{P}(\tau \mid \pi, \mu)Q(s_t,a_t)\boldsymbol{\nabla}\log\pi(a_t \mid s_t)\,\mathrm{d}\tau \tag{43}$$

$$= \sum_{t=0}^{\infty}\gamma^t \mathbb{E}_{\tau\sim P}[Q(s_t,a_t)\boldsymbol{\nabla}\log\pi(a_t \mid s_t)] \tag{44}$$

$$= \mathbb{E}_{\tau\sim P}\left[\sum_{t=0}^{\infty}\gamma^t Q(s_t,a_t)\boldsymbol{\nabla}\log\pi(a_t \mid s_t)\right] \tag{45}$$

■

2 Relationship between soft Q-learning and policy gradient

Entropy-regularized reinforcement learning is widely used in reality, especially in recent advances of LLM post-training. Instead of the traditional discounted return $\sum_{t=0}^{\infty} \gamma^t r_t$, we consider the following entropy-regularized return function: given a reference policy $\overline{\pi}$, the total return is defined as

$$\sum_{t=0}^{\infty} \gamma^t \{r(s_t, a_t) - \tau D_{\mathrm{KL}}(\pi(\cdot \mid s_t) \,\|\, \overline{\pi}(\cdot \mid s_t))\}, \tag{46}$$

where $D_{\mathrm{KL}}(p \,\|\, \overline{p}) := \sum_a p(a) \log(p(a)/\overline{p}(a))$ denotes the KL-divergence between two distributions, and characterizes the difference between these two distributions. Hence maximizing the entropy-regularized return Equation 46 not only aims for maximizing the total return of policy π, but also keeps the policy π close to the reference policy $\overline{\pi}$. We define the value function of policy π at state s to be the expected entropy-regularized return:

$$V_\pi(s) := \mathbb{E}\left[\sum_{t=0}^{\infty} \{\gamma^t r(s_t, a_t) - \tau D_{\mathrm{KL}}(\pi(\cdot \mid s_t) \,\|\, \overline{\pi}(\cdot \mid s_t))\} \mid s_0 = s\right], \tag{47}$$

and similarly the Q-function of policy π:

$$Q_\pi(s, a) := \mathbb{E}\left[r(s_0, a_0) + \sum_{t=1}^{\infty} \gamma^t \{r(s_t, a_t) - \tau D_{\mathrm{KL}}(\pi(\cdot \mid s_t) \,\|\, \overline{\pi}(\cdot \mid s_t))\} \mid s_0 = s, a_0 = a\right]. \tag{48}$$

This gives the relationship

$$V_\pi(s) = \mathbb{E}_{a \sim \pi(\cdot \mid s)}[Q_\pi(s, a)] - \tau D_{\mathrm{KL}}(\pi(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)). \tag{49}$$

Note: τ is the temperature parameter, not trajectory. $\tau = 0 \Rightarrow$ greedy. $\tau \to \infty \Rightarrow$ reference policy. Used to weight the KL-divergence cost.

2.1 Part 1: Boltzman policy

Given a Q-function, show that the optimizer of the optimization problem

$$\arg\max_{\pi}\Big\{\mathbb{E}_{a\sim\pi(\cdot\mid s)}[Q_\pi(s,a)]-\tau D_{\mathrm{KL}}(\pi(\cdot\mid s)\,\|\,\overline{\pi}(\cdot\mid s))\Big\} \tag{50}$$

over all policies satisfies

$$\pi_Q^{\mathcal{B}}(a\mid s)=\overline{\pi}(a\mid s)\cdot\frac{\exp(Q(s,a)/\tau)}{\mathbb{E}_{a\sim\overline{\pi}(\cdot\mid s)}[\exp(Q(s,a)/\tau)]},\quad \forall a\in\mathcal{A}. \tag{51}$$

The policy $\pi_Q^{\mathcal{B}}$ is called the Boltzman policy.

Expand the optimized expression

$$\mathbb{E}_{a\sim\pi(\cdot\mid s)}[Q_\pi(s,a)]=\sum_a \pi(a\mid s)Q_\pi(s,a) \tag{52}$$

$$D_{\mathrm{KL}}(\pi(\cdot\mid s)\,\|\,\overline{\pi}(\cdot\mid s))=\sum_a \pi(a\mid s)\log\left(\frac{\pi(a\mid s)}{\overline{\pi}(a\mid s)}\right) \tag{53}$$

Combining:

$$\mathbb{E}_{a\sim\pi(\cdot\mid s)}[Q_\pi(s,a)]-\tau D_{\mathrm{KL}}(\pi(\cdot\mid s)\,\|\,\overline{\pi}(\cdot\mid s)) \tag{54}$$

$$=\sum_a \pi(a\mid s)Q_\pi(s,a)-\tau\sum_a \pi(a\mid s)\log\left(\frac{\pi(a\mid s)}{\overline{\pi}(a\mid s)}\right) \tag{55}$$

$$=\sum_a \pi(a\mid s)\left(Q_\pi(s,a)-\tau\log\left(\frac{\pi(a\mid s)}{\overline{\pi}(a\mid s)}\right)\right) \tag{56}$$

We will assume a fixed Q, independent from π. Maximizing this quantity is a constrained optimization problem below:

$$\max\quad \sum_a \pi(a\mid s)\left(Q_\pi(s,a)-\tau\log\left(\frac{\pi(a\mid s)}{\overline{\pi}(a\mid s)}\right)\right) \tag{57}$$

$$\text{w.r.t}\quad \sum_a \pi(a\mid s)=1 \tag{58}$$

$$\mathcal{L}=\sum_a \pi(a\mid s)\left(Q(s,a)-\tau\log\left(\frac{\pi(a\mid s)}{\overline{\pi}(a\mid s)}\right)\right)+\lambda\left(1-\sum_a \pi(a\mid s)\right) \tag{59}$$

$$=\sum_a \pi(a\mid s)(Q(s,a)-\tau\log\pi(a\mid s)+\tau\log\overline{\pi}(a\mid s))+\lambda\left(1-\sum_a \pi(a\mid s)\right) \tag{60}$$

Abusing notation by dropping most arguments for simplicity: (61)

$$=\sum_a \pi(Q-\tau\log\pi+\tau\log\overline{\pi})+\lambda\left(1-\sum_a \pi(a\mid s)\right) \tag{62}$$

Lets take the derivative of the policy at some fixed action a and state s. Only one element from each of the sums will not be a constant:

$$\frac{\partial \mathcal{L}}{\partial \pi(a \mid s)} = 0 = \left[\pi\left(\frac{\tau}{\pi}\right) + 1(Q - \tau \log \pi + \tau \log \overline{\pi})\right] + \lambda(-1) \tag{63}$$

$$= [\tau + Q - \tau \log \pi + \tau \log \overline{\pi}] - \lambda \tag{64}$$

$$= Q + \tau(1 - \log \pi + \log \overline{\pi}) - \lambda \tag{65}$$

Solving for π:

$$\lambda - Q = \tau(1 - \log \pi + \log \overline{\pi}) \tag{66}$$

$$\Rightarrow \frac{\lambda - Q}{\tau} - 1 - \log \overline{\pi} = -\log \pi \tag{67}$$

$$\Rightarrow \log \pi = \frac{Q - \lambda}{\tau} + 1 + \log \overline{\pi} \tag{68}$$

$$\Rightarrow \pi = e^{\frac{Q-\lambda}{\tau} + 1 + \log \overline{\pi}} \tag{69}$$

$$= \overline{\pi} \exp\left(\frac{Q - \lambda + \tau}{\tau}\right) \tag{70}$$

$$= \overline{\pi} \exp\left(\frac{Q}{\tau}\right) \exp\left(\frac{\tau - \lambda}{\tau}\right) \tag{71}$$

$$= \overline{\pi} \exp\left(\frac{Q}{\tau}\right) \exp\left(\frac{\tau - \lambda}{\tau}\right) \tag{72}$$

Sub-Solution

We can now introduce the probability constraint:

$$1 = \sum_a \pi(a \mid s) = \sum_a \overline{\pi}(a \mid s) \exp\left(\frac{Q(s,a)}{\tau}\right) \exp\left(\frac{\tau - \lambda}{\tau}\right) \tag{73}$$

And we can solve for $\exp\left(\frac{\tau-\lambda}{\tau}\right)$, the constant:

$$1 = \exp\left(\frac{\tau - \lambda}{\tau}\right) \sum_a \overline{\pi}(a \mid s) \exp\left(\frac{Q(s,a)}{\tau}\right) \tag{74}$$

$$\Rightarrow \exp\left(\frac{\tau - \lambda}{\tau}\right) = \frac{1}{\sum_a \overline{\pi}(a \mid s) \exp\left(\frac{Q(s,a)}{\tau}\right)} \tag{75}$$

Plugging this term in:

$$\pi = \overline{\pi} \exp\left(\frac{Q}{\tau}\right) \exp\left(\frac{\tau - \lambda}{\tau}\right) \tag{76}$$

$$= \overline{\pi} \frac{\exp\left(\frac{Q}{\tau}\right)}{\sum_a \overline{\pi}(a \mid s) \exp\left(\frac{Q(s,a)}{\tau}\right)} \tag{77}$$

This provides us with the Boltzman policy:

$$\pi_Q^{\mathcal{B}}(a \mid s) = \overline{\pi}(a \mid s) \cdot \frac{\exp(Q(s,a)/\tau)}{\mathbb{E}_{a \sim \overline{\pi}(\cdot \mid s)}[\exp(Q(s,a)/\tau)]}, \quad \forall a \in \mathcal{A}. \tag{78}$$

2.2 Part 2: Value function representation

Given a Q-function, show that the value function of the Boltzman policy has the following representation:

$$V_{\pi_Q^B}(s) = \tau \log \mathbb{E}_{a \sim \overline{\pi}(\cdot \mid s)} \left[\exp\left(\frac{Q(s,a)}{\tau} \right) \right]. \tag{79}$$

We simply need to plug in the Boltzman policy $\pi = \pi_Q^{\mathcal{B}}$ into equation for V_π and letting Q_π instead be some arbitrary Q:

$$V_\pi(s) = \mathbb{E}_{a \sim \pi(\cdot \mid s)}[Q(s,a)] - \tau D_{\mathrm{KL}}(\pi(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)). \tag{80}$$

And letting the constant denominator be $C := \mathbb{E}_{a \sim \overline{\pi}(\cdot \mid s)}[\exp(Q(s,a)/\tau)]$

$$\pi_Q^{\mathcal{B}}(a \mid s) = \overline{\pi}(a \mid s) \cdot \frac{\exp(Q(s,a)/\tau)}{\mathbb{E}_{a \sim \overline{\pi}(\cdot \mid s)}[\exp(Q(s,a)/\tau)]}, \quad \forall a \in \mathcal{A} \tag{81}$$

$$= \frac{1}{C} \overline{\pi}(a \mid s) \exp(Q(s,a)/\tau), \quad \forall a \in \mathcal{A}. \tag{82}$$

Starting with the **first term**:

$$\mathbb{E}_{a \sim \pi(\cdot \mid s)}[Q(s,a)] = \sum_a \pi(a \mid s) \cdot Q(s,a) \tag{83}$$

$$= \sum_a \overline{\pi}(a \mid s) \cdot \frac{\exp(Q(s,a)/\tau)}{\mathbb{E}_{a \sim \overline{\pi}(\cdot \mid s)}[\exp(Q(s,a)/\tau)]} \cdot Q(s,a) \tag{84}$$

For simplicity lets call this constant denominator C

$$= \sum_a \overline{\pi}(a \mid s) \cdot \frac{\exp(Q(s,a)/\tau)}{C} \cdot Q(s,a) \tag{85}$$

$$= \frac{1}{C} \sum_a \overline{\pi}(a \mid s) Q(s,a) \exp\left(\frac{Q(s,a)}{\tau} \right) \tag{86}$$

For the **second term**, KL-divergence:

$$D_{\mathrm{KL}}(\pi(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)) = \sum_a \pi(a \mid s) \log \frac{\pi(a \mid s)}{\overline{\pi}(a \mid s)} \tag{87}$$

$$= \sum_a \frac{1}{C} \overline{\pi}(a \mid s) \exp(Q(s,a)/\tau) \left[\log\left(\frac{1}{C} \overline{\pi}(a \mid s) \exp(Q(s,a)/\tau) \right) - \log \overline{\pi}(a \mid s) \right] \tag{88}$$

$$= \frac{1}{C} \sum_a \overline{\pi}(a \mid s) \exp(Q(s,a)/\tau) \left[\log \frac{1}{C} + \log \overline{\pi}(a \mid s) + Q(s,a)/\tau - \log \overline{\pi}(a \mid s) \right] \tag{89}$$

$$= \frac{1}{C} \sum_a \overline{\pi}(a \mid s) \exp(Q(s,a)/\tau) \left[\log \frac{1}{C} + Q(s,a)/\tau \right] \tag{90}$$

$$\tag{91}$$

$$= \frac{\log(1/C)}{C} \sum_a \overline{\pi}(a \mid s) \exp\left(\frac{Q(s,a)}{\tau} \right) \tag{92}$$

$$+\frac{1}{C\tau}\sum_a \overline{\pi}(a \mid s)\exp\left(\frac{Q(s,a)}{\tau}\right)Q(s,a) \tag{93}$$

$$\tag{94}$$

$$\tag{95}$$

$$= \log(1/C) + \frac{1}{C\tau}\sum_a \overline{\pi}(a \mid s)\exp\left(\frac{Q(s,a)}{\tau}\right)Q(s,a) \tag{96}$$

Combining the two terms:

$$V_\pi(s) = \left[\frac{1}{C}\sum_a \overline{\pi}(a \mid s)Q(s,a)\exp\left(\frac{Q(s,a)}{\tau}\right)\right] \tag{97}$$

$$-\tau\left[\log(1/C) + \frac{1}{C\tau}\sum_a \overline{\pi}(a \mid s)\exp\left(\frac{Q(s,a)}{\tau}\right)Q(s,a)\right] \tag{98}$$

$$\tag{99}$$

$$= \left[\left(\frac{1}{C} - \tau\frac{1}{C\tau}\right)\sum_a \overline{\pi}(a \mid s)Q(s,a)\exp\left(\frac{Q(s,a)}{\tau}\right)\right] - \tau\log(1/C) \tag{100}$$

$$\tag{101}$$

$$= -\tau\log\left(\frac{1}{C}\right) \tag{102}$$

$$= \tau\log C \tag{103}$$

$$= \tau\log\mathbb{E}_{a\sim\overline{\pi}(\cdot \mid s)}\left[\exp\left(\frac{Q(s,a)}{\tau}\right)\right] \tag{104}$$

This gives us the expected:

$$V_{\pi_Q^B}(s) = \tau\log\mathbb{E}_{a\sim\overline{\pi}(\cdot \mid s)}\left[\exp\left(\frac{Q(s,a)}{\tau}\right)\right]. \tag{105}$$

■

2.3 Part 3: Single-Step Soft Q-learning

In the following, we parametrize the policy Q-function Q_θ by parameter θ, and we denote the Boltzman policy $\pi^B_{Q_\theta}$ by π_θ, and its value function $V_{\pi^B_{Q_\theta}}$ as V_θ. We let

$$y_t = r_t + \gamma V_{\pi^B_Q}(s_{t+1}). \tag{106}$$

Show that the gradient of the soft Q-learning objective at a single sample can be written as

$$\nabla_\theta \left[\frac{1}{2}\|Q_\theta(s_t, a_t) - y_t\|^2\right] \tag{107}$$

$$= (\tau \nabla_\theta \log \pi_\theta(a_t \mid s_t) + \nabla_\theta V_\theta(s_t)) \tag{108}$$

$$\cdot \left(\tau \cdot \log \frac{\pi_\theta(a_t \mid s_t)}{\overline{\pi}(a_t \mid s_t)} - \tau D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s_t) \,\|\, \overline{\pi}(\cdot \mid s_t)) - \delta_t\right), \tag{109}$$

where δ_t is defined as

$$\delta_t = y_t - (\tau D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s_t) \,\|\, \overline{\pi}(\cdot \mid s_t)) + V_\theta(s_t)). \tag{110}$$

Some simplification first:

$$\nabla\left[\frac{1}{2}\|Q(s_t, a_t) - y_t\|^2\right] = \frac{1}{2}\nabla(Q(s_t, a_t) - y_t)^2 \tag{111}$$

$$= (Q(s_t, a_t) - y_t)\nabla(Q(s_t, a_t) - y_t) \tag{112}$$

$$= (Q(s_t, a_t) - y_t)\left(\nabla Q(s_t, a_t) - \underbrace{\nabla y_t}_{0 \text{ according to piazza}}\right) \tag{113}$$

$$= (Q(s_t, a_t) - y_t)\nabla Q(s_t, a_t) \tag{114}$$

Expanding: y_t

$$y_t = r_t + \gamma V_{\pi^B_Q}(s_{t+1}) \tag{115}$$

$$= r_t + \gamma\tau \log \mathbb{E}_{a \sim \overline{\pi}(\cdot \mid s)}\left[\exp\left(\frac{Q(s, a)}{\tau}\right)\right] \tag{116}$$

$$= r_t + \gamma\tau \log \sum_a \overline{\pi}(a \mid s) \exp\left(\frac{Q(s, a)}{\tau}\right) \tag{117}$$

We want to eliminate ∇Q and replace it with ∇V. We can do this through the definition given above:

$$V_\theta(s) = \mathbb{E}_{a \sim \pi_\theta(\cdot \mid s)}[Q_\theta(s, a)] - \tau D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)) \tag{118}$$

$$\nabla V_\theta(s) = \nabla \mathbb{E}_{a \sim \pi_\theta(\cdot \mid s)}[Q_\theta(s, a)] - \nabla \tau D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)) \tag{119}$$

$$= \nabla \sum_a \pi_\theta(a \mid s) Q_\theta(s, a) - \tau \nabla D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)) \tag{120}$$

Focusing on the **first term**:

$$\nabla \sum_a \pi_\theta(a \mid s) Q_\theta(s, a) = \sum_a [\pi_\theta(a \mid s) \nabla Q_\theta(s, a) + Q_\theta(a \mid s) \nabla \pi_\theta(s, a)] \tag{121}$$

$$= \sum_a \pi_\theta(a \mid s)[\nabla Q_\theta(s,a) + Q_\theta(a \mid s)\nabla \log \pi_\theta(s,a)] \tag{122}$$

And for the **second term**:

$$\nabla D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)) = \sum_a \nabla \left(\pi_\theta(a \mid s) \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \right) \tag{123}$$

$$= \sum_a \left(\pi_\theta(a \mid s) \nabla \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \nabla \pi_\theta(a \mid s) \right) \tag{124}$$

$$= \sum_a \left(\pi_\theta(a \mid s)[\nabla \log(\pi_\theta(a \mid s)) - \nabla \log(\overline{\pi}(a \mid s))] + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \nabla \pi_\theta(a \mid s) \right) \tag{125}$$

$$= \sum_a \left(\pi_\theta(a \mid s)[\nabla \log \pi_\theta(a \mid s)] + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \nabla \pi_\theta(a \mid s) \right) \tag{126}$$

$$= \sum_a \left(\pi_\theta(a \mid s)[\nabla \log \pi_\theta(a \mid s)] + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \pi_\theta(a \mid s) \nabla \log \pi_\theta(a \mid s) \right) \tag{127}$$

$$= \sum_a \pi_\theta(a \mid s) \left(\nabla \log \pi_\theta(a \mid s) + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \nabla \log \pi_\theta(a \mid s) \right) \tag{128}$$

$$= \sum_a \pi_\theta(a \mid s) \nabla \log \pi_\theta(a \mid s) \left(1 + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \right) \tag{129}$$

Combining

$$\nabla V_\theta(s) = \nabla \sum_a \pi_\theta(a \mid s) Q_\theta(s,a) - \tau \nabla D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, \overline{\pi}(\cdot \mid s)) \tag{130}$$

$$\tag{131}$$

$$= \sum_a \pi_\theta(a \mid s)[\nabla Q_\theta(s,a) + Q_\theta(a \mid s)\nabla \log \pi_\theta(s,a)] \tag{132}$$

$$-\tau \left[\sum_a \pi_\theta(a \mid s) \nabla \log \pi_\theta(a \mid s) \left(1 + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \right) \right] \tag{133}$$

$$\tag{134}$$

$$= \sum_a \pi_\theta(a \mid s) \Bigg[\tag{135}$$

$$\nabla Q_\theta(s,a) + Q_\theta(a \mid s) \nabla \log \pi_\theta(s,a) - \tau \left(\nabla \log \pi_\theta(a \mid s) \left(1 + \log \frac{\pi_\theta(a \mid s)}{\overline{\pi}(a \mid s)} \right) \right) \tag{136}$$

$$\Bigg] \tag{137}$$

Final Steps

1. And then we can solve for ∇Q_θ to only have an expression involving $\nabla V_\theta, \nabla \log \pi_\theta$, and Q_θ.
2. Substitute Q_θ from the Boltzmann policy definition
3. Pull δ_t into a separate term

2.4 Part 4: Relationship between soft Q-learning and policy gradient

If we assume that the samples s_t, a_t, r_t are all collected according to policy π, show the following relationship between the gradient of soft Q-learning and the policy gradient.

$$\underbrace{\nabla_\theta \mathbb{E}_\pi \left[\frac{1}{2} \| Q_\theta(s_t, a_t) - y_t \|^2 \right] \mid_{\pi=\pi_\theta}}_{\text{Q-learning gradient}} \tag{138}$$

$$= \underbrace{\tau \cdot \mathbb{E}_\pi [-\nabla_\theta \pi_\theta(a_t \mid s_t) \delta_t + \tau \nabla_\theta D_{\mathrm{KL}}(\pi(\cdot \mid s_t) \| \overline{\pi}(\cdot \mid s_t))] \mid_{\pi=\pi_\theta}}_{\text{policy gradient}} \tag{139}$$

$$+ \underbrace{\frac{1}{2} \nabla_\theta \mathbb{E}_\pi \left[\left\| V_\theta(s_t) - \hat{V}_t \right\|^2 \right] \mid_{\pi=\pi_\theta}}_{\text{value function gradient}}, \tag{140}$$

where $\hat{V}_t$ is defined as

$$\hat{V}_t = y_t - \tau D_{\mathrm{KL}}(\pi(\cdot \mid s_t) \| \overline{\pi}(\cdot \mid s_t)) \mid_{\pi=\pi_\theta}. \tag{141}$$

To solve

1. Start with result from part three above
2. Expand terms
3. Use $\delta_t = \hat{V}_t - V_\theta(s_t)$
4. Use definition of the norm and expectation to put into required form.

3 REINFORCE with cartpole

In this problem, we will try solving the cart-pole problem via a policy space method using neural networks. The setting is as follows. A pole is attached by an un-actuated joint to a cart, which moves along a frictionless track. The pendulum is placed upright on the cart and the goal is to balance the pole by applying forces in the left and right direction on the cart.

- **Action space.** $\{0, 1\}$, where 0 means the cart is pushed to the left, and 1 means the cart is pushed to the right.
- **State space.** (Cart Position, Cart Velocity, Pole Angle, Pole Angular Velocity). In particular,
  - The cart position can take values between $(-4.8, 4.8)$, but the episode terminates if the cart leaves the $(-2.4, 2.4)$ range.
  - The pole angle can be observed between $(-0.418, 0.418)$ radians (or $\pm 24°$), but the episode terminates if the pole angle is not in the range $(-0.2095, 0.2095)$ (or $\pm 12°$). The starting pole angle is sampled uniformly from $(-0.05, 0.05)$.
- **Rewards.** Our goal is to keep the pole upright for as long as possible. So a reward of "$+1$" for every step taken, including the termination step, is allotted.
- **Episode Termination.** The episode terminates if any one of the following occurs:
  - Pole Angle is greater than $\pm 12°$.
  - Cart Position is greater than ± 2.4 (center of the cart reaches the edge of the display).
  - Episode length is greater than 500.

Please read the detailed instructions in the Colab link (can be found here). Complete the code, do the experiments, and answer the questions in the notebook. For submission, please include all your results in the pdf submitted to Gradescope, as well as a link with your used code.

HW01

In [1]:

```python
# Configure Plotly to render static PNGs in this notebook
import plotly.io as pio

# Produce PNG outputs instead of interactive HTML
pio.renderers.default = "png"

# If you want interactive HTML instead, comment the line above and uncomment below:
# pio.renderers.default = "notebook_connected"

# Optional: improve PNG resolution when Kaleido is available
pio.defaults.default_scale = 2
pio.defaults.default_width = 900
pio.defaults.default_height = 600
```

In [28]:

```python
# imports
import numpy as np
import plotly.graph_objects as go
```

In [17]:

```python
# Settings for the LQR
T = 20
A = np.array([[1, 1], [0, 1]])
B = np.array([[0], [1]])
x_0 = np.array([[5], [0]])

position_penalty = [(np.array([[q, 0], [0, 1]]), 1, np.identity(2)) for q in (0.1, 10)]
control_penalty = [(np.identity(2), r, np.identity(2)) for r in (0.1, 10)]
terminal_penalty = [(np.identity(2), 1, np.identity(2) * q_f) for q_f in (3, 10)]
benchmark_penalty = [(np.identity(2), 1, np.identity(2) * 1)]

penalty = {
    "position": position_penalty,
    "control": control_penalty,
    "terminal": terminal_penalty,
    "benchmark": benchmark_penalty,
}


def dynamics(x_prev: np.ndarray, u: np.ndarray) -> np.ndarray:
    return A @ x_prev + B @ u
```

In [14]:

```python
def P(
    T: int,
    A: np.ndarray,
```

```
    B: np.ndarray,
    Q: np.ndarray,
    Q_f: np.ndarray,
    r: float,
    t: int = 0,
) -> list[np.ndarray | None]:
    """Calculate p"""
    p = [None for _ in range(T + 1)]
    R = np.array([[r]])

    def P_dp(t: int) -> np.ndarray:
        # Already in list
        if p[t] is not None:
            return p[t]

        # Base Case
        if t == T:
            p[T] = Q_f
            return p[t]

        # Recursive case, calculate
        inv = np.linalg.inv(B.T @ P_dp(t + 1) @ B + R)
        p[t] = A.T @ (P_dp(t + 1) - P_dp(t + 1) @ B @ inv @ B.T @ P_dp(t + 1)) @ A + Q
        return p[t]

    # Fill the table
    P_dp(t)
    return p


Q, r, Q_f = position_penalty[1]
p = P(T, A, B, Q, Q_f, r)
for x in p:
    print(x)
```

```txt
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
```

```
[[2.94712297 2.36920541]
 [2.36920541 4.61313426]]
[[2.94712296 2.3692054 ]
 [2.3692054  4.61313425]]
[[2.94712294 2.36920537]
 [2.36920537 4.6131342 ]]
[[2.94712279 2.36920511]
 [2.36920511 4.61313372]]
[[2.94712153 2.36920309]
 [2.36920309 4.61313036]]
[[2.9471134  2.36919108]
 [2.36919108 4.61311203]]
[[2.94707316 2.36913678]
 [2.36913678 4.61303492]]
[[2.94691724 2.36894164]
 [2.36894164 4.61275964]]
[[2.94634598 2.36817761]
 [2.36817761 4.61147086]]
[[2.94300518 2.3626943 ]
 [2.3626943  4.60103627]]
[[2.91428571 2.31428571]
 [2.31428571 4.51428571]]
[[2.71428571 2.        ]
 [2.         4.        ]]
[[2.  1. ]
 [1.  2.5]]
[[1. 0.]
 [0. 1.]]
```

In [15]:

Python

```python
def L(
    T: int,
    A: np.ndarray,
    B: np.ndarray,
    Q: np.ndarray,
    Q_f: np.ndarray,
    r: float,
    t: int = 0,
) -> list[np.ndarray | None]:
    """Calculate p"""
    p = P(T, A, B, Q, Q_f, r, t=t)
    l = [None for _ in range(T)]
    R = np.array([[r]])

    # Note: not actually recursive on L.
    def L_dp(t: int) -> np.ndarray:
        # Already in list
        if l[t] is not None:
            return l[t]
```

```
20
21          # Recursive case, calculate
22          inv = np.linalg.inv(B.T @ p[t + 1] @ B + R)
23          l[t] = -inv @ B.T @ p[t + 1] @ A
24          return l[t]
25
26      # Fill the table
27      for i in range(T):
28          L_dp(i)
29      return l
30
31
32  Q, r, Q_f = position_penalty[1]
33  l = L(T, A, B, Q, Q_f, r)
34  for x in l:
35      print(x)
```

```txt
1   [[-0.42208244 -1.24392885]]
2   [[-0.42208244 -1.24392885]]
3   [[-0.42208244 -1.24392885]]
4   [[-0.42208244 -1.24392885]]
5   [[-0.42208244 -1.24392885]]
6   [[-0.42208244 -1.24392885]]
7   [[-0.42208244 -1.24392885]]
8   [[-0.42208244 -1.24392885]]
9   [[-0.42208244 -1.24392885]]
10  [[-0.42208243 -1.24392882]]
11  [[-0.42208232 -1.24392861]]
12  [[-0.42208156 -1.24392727]]
13  [[-0.42207768 -1.24392094]]
14  [[-0.42206362 -1.24389814]]
15  [[-0.4220244 -1.243818 ]]
16  [[-0.42183164 -1.24329325]]
17  [[-0.41968912 -1.23834197]]
18  [[-0.4 -1.2]]
19  [[-0.28571429 -1.        ]]
20  [[ 0.  -0.5]]
```

In [ ]:

In [ ]:

```python
1   from collections.abc import Callable
2
3
4   def simulate_optimal(
5       T: int,
6       A: np.ndarray,
7       B: np.ndarray,
8       Q: np.ndarray,
```

```
    Q_f: np.ndarray,
    r: float,
    x_0: np.ndarray,
    dynamics: Callable,
) → tuple[list[np.ndarray | None], list[np.ndarray | None]]:
    """Calculate u, the controls"""
    l = L(T, A, B, Q, Q_f, r, t=0)
    u_optimal = [None for _ in range(T)]
    x_optimal = [None for _ in range(T + 1)]
    x_optimal[0] = x_0

    # Take actions for each
    for i in range(T):
        u_optimal[i] = l[i] @ x_optimal[i]
        x_optimal[i + 1] = dynamics(x_optimal[i], u_optimal[i])

    return x_optimal, u_optimal


Q, r, Q_f = position_penalty[1]
x_optimal, u_optimal = simulate_optimal(T, A, B, Q, Q_f, r, x_0, dynamics)
for i, u in enumerate(u_optimal):
    print(f"\n\nFor t = {i}")
    print("x_t =\n", x_optimal[i])
    print("u_t =\n", u)
# Last state not printed
print(f"For t = {T}")
print(x[-1])
```

```txt

 For t = 0
 x_t =
  [[5]
  [0]]
 u_t =
  [[-3.88075109]]


 For t = 1
 x_t =
  [[ 5.        ]
  [-3.88075109]]
 u_t =
  [[2.77826581]]


 For t = 2
 x_t =
```

```
 [[ 1.11924891]
 [-1.10248528]]
u_t =
 [[1.02305947]]


For t = 3
x_t =
 [[ 0.01676362]
 [-0.07942581]]
u_t =
 [[0.1232764]]


For t = 4
x_t =
 [[-0.06266219]
 [ 0.04385059]]
u_t =
 [[-0.02660836]]


For t = 5
x_t =
 [[-0.0188116 ]
 [ 0.01724223]]
u_t =
 [[-0.01498547]]


For t = 6
x_t =
 [[-0.00156937]
 [ 0.00225676]]
u_t =
 [[-0.00265432]]


For t = 7
x_t =
 [[ 0.00068738]
 [-0.00039756]]
u_t =
 [[0.00014867]]


For t = 8
x_t =
 [[ 0.00028982]
```

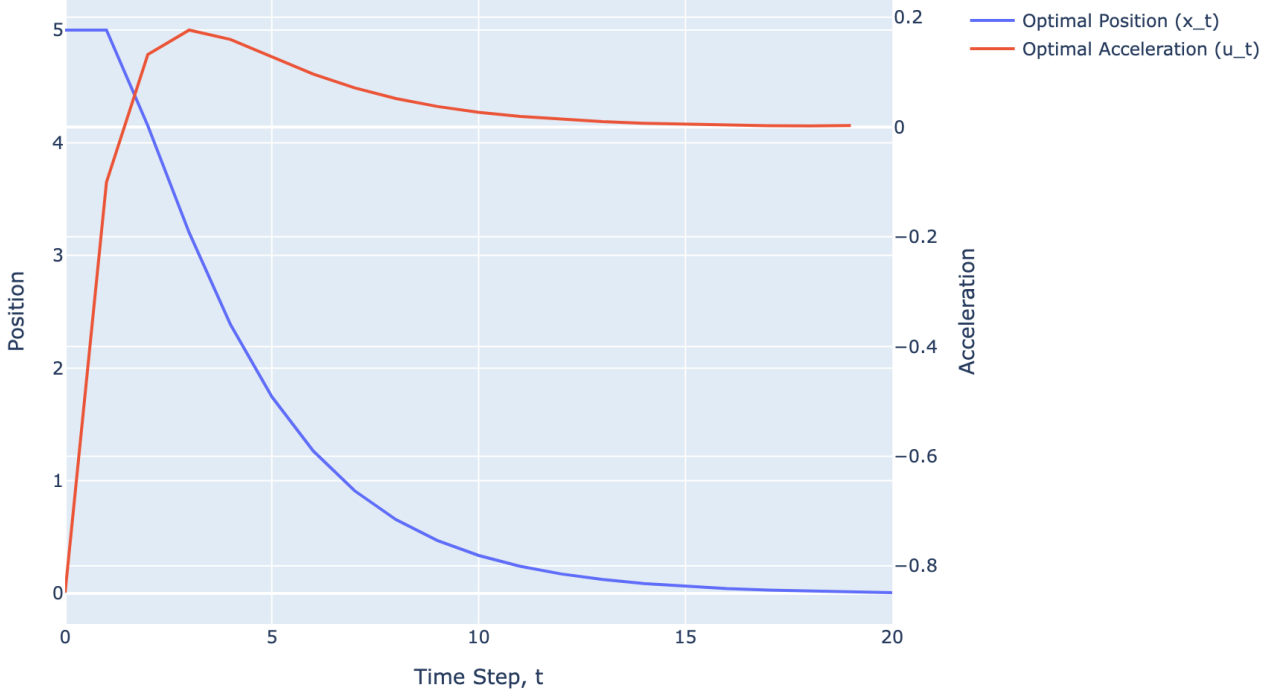
```
 [-0.00024889]]
u_t =
 [[0.00020213]]


For t = 9
x_t =
 [[ 4.09264846e-05]
 [-4.67586623e-05]]
u_t =
 [[4.84685237e-05]]


For t = 10
x_t =
 [[-5.83217774e-06]
 [ 1.70986137e-06]]
u_t =
 [[1.592679e-06]]


For t = 11
x_t =
 [[-4.12231637e-06]
 [ 3.30254037e-06]]
u_t =
 [[-2.46732299e-06]]


For t = 12
x_t =
 [[-8.19776001e-07]
 [ 8.35217383e-07]]
u_t =
 [[-7.96887948e-07]]


For t = 13
x_t =
 [[1.54413813e-08]
 [3.83294349e-08]]
u_t =
 [[-7.77546698e-08]]


For t = 14
x_t =
 [[ 5.37708162e-08]
 [-3.94252349e-08]]
```

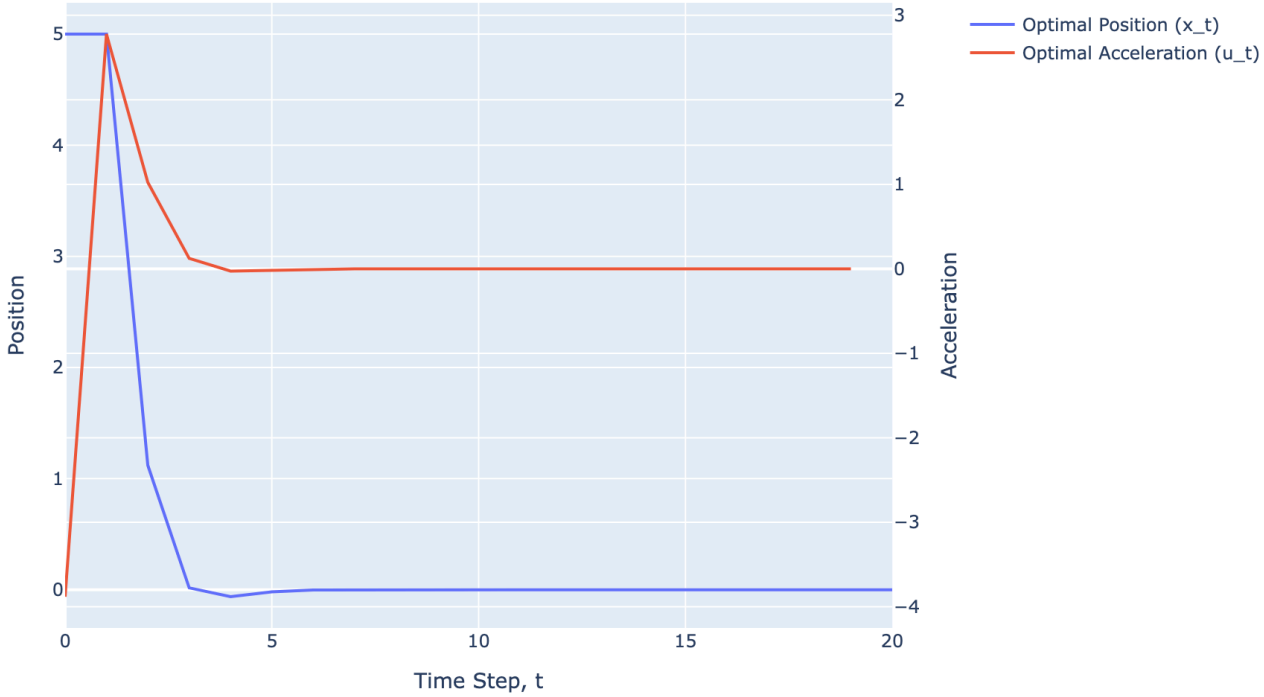
```
u_t =
 [[2.59157761e-08]]


For t = 15
x_t =
 [[ 1.43455813e-08]
 [-1.35094589e-08]]
u_t =
 [[1.20461853e-08]]


For t = 16
x_t =
 [[ 8.36122425e-10]
 [-1.46327360e-09]]
u_t =
 [[1.86155542e-09]]


For t = 17
x_t =
 [[-6.27151179e-10]
 [ 3.98281818e-10]]
u_t =
 [[-1.9000719e-10]]


For t = 18
x_t =
 [[-2.28869361e-10]
 [ 2.08274628e-10]]
u_t =
 [[-1.42883382e-10]]


For t = 19
x_t =
 [[-2.05947332e-11]
 [ 6.53912460e-11]]
u_t =
 [[-3.2695623e-11]]
For t = 20
[ 0.  -0.5]
```

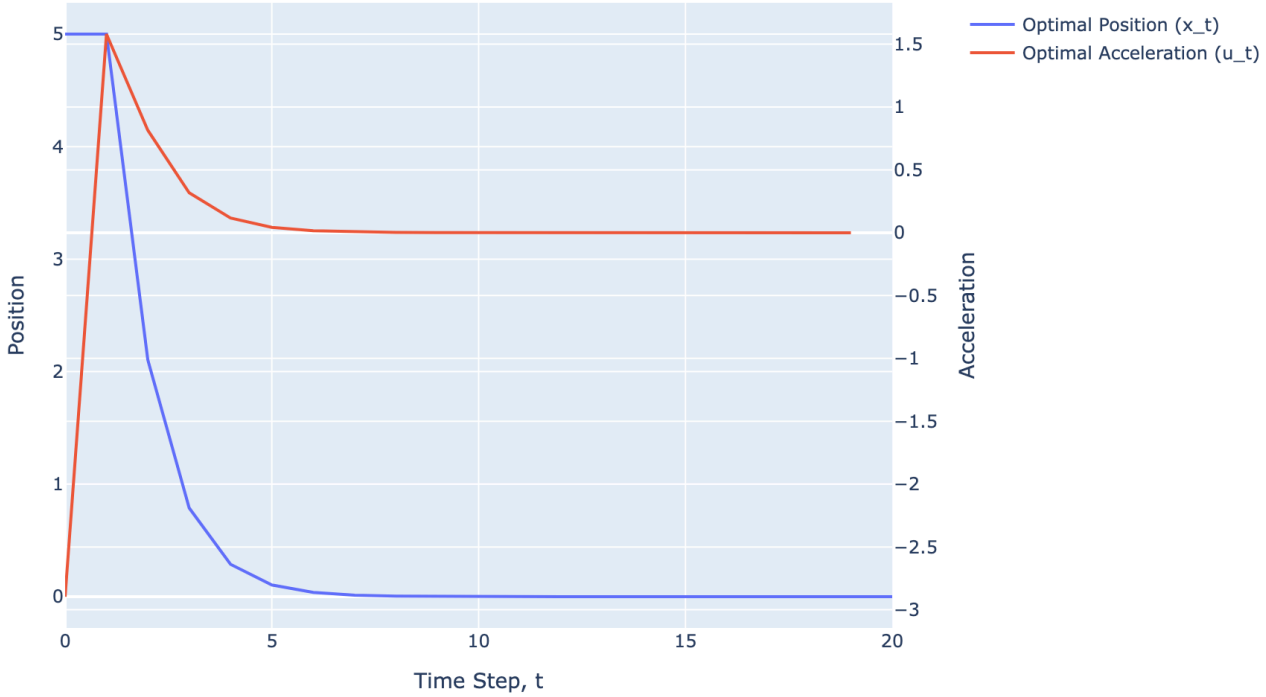
In [ ]:

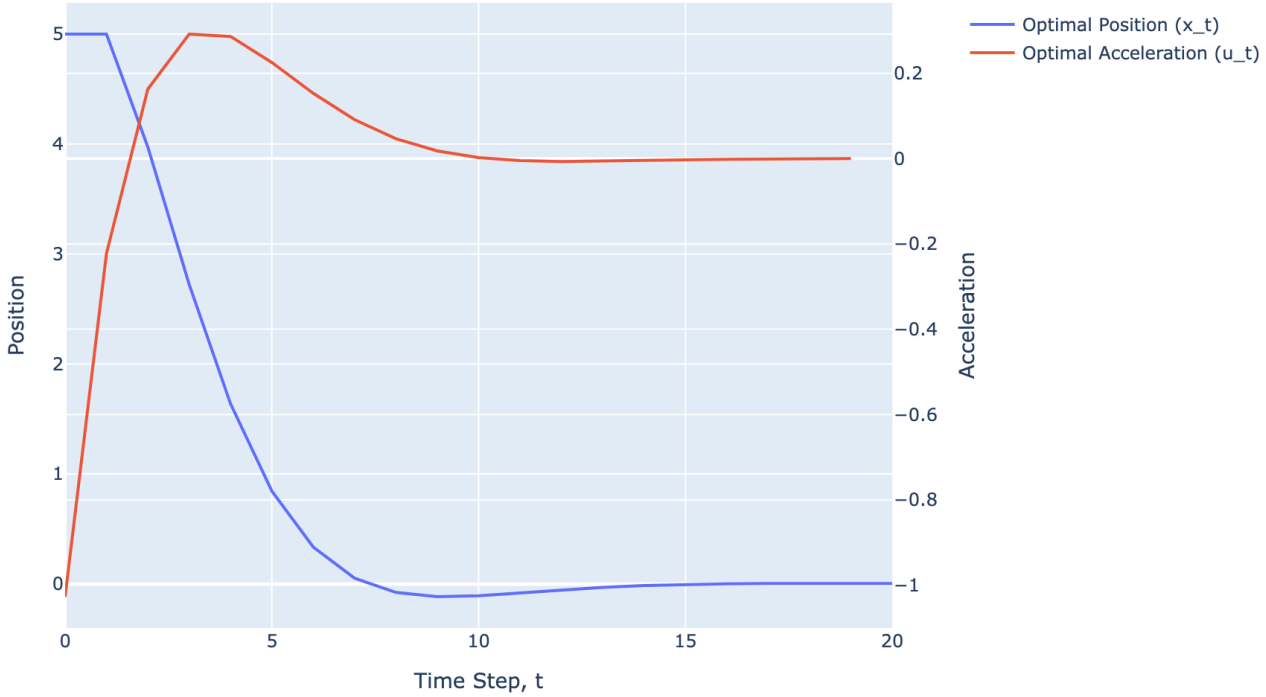
```python
from plotly.subplots import make_subplots


```

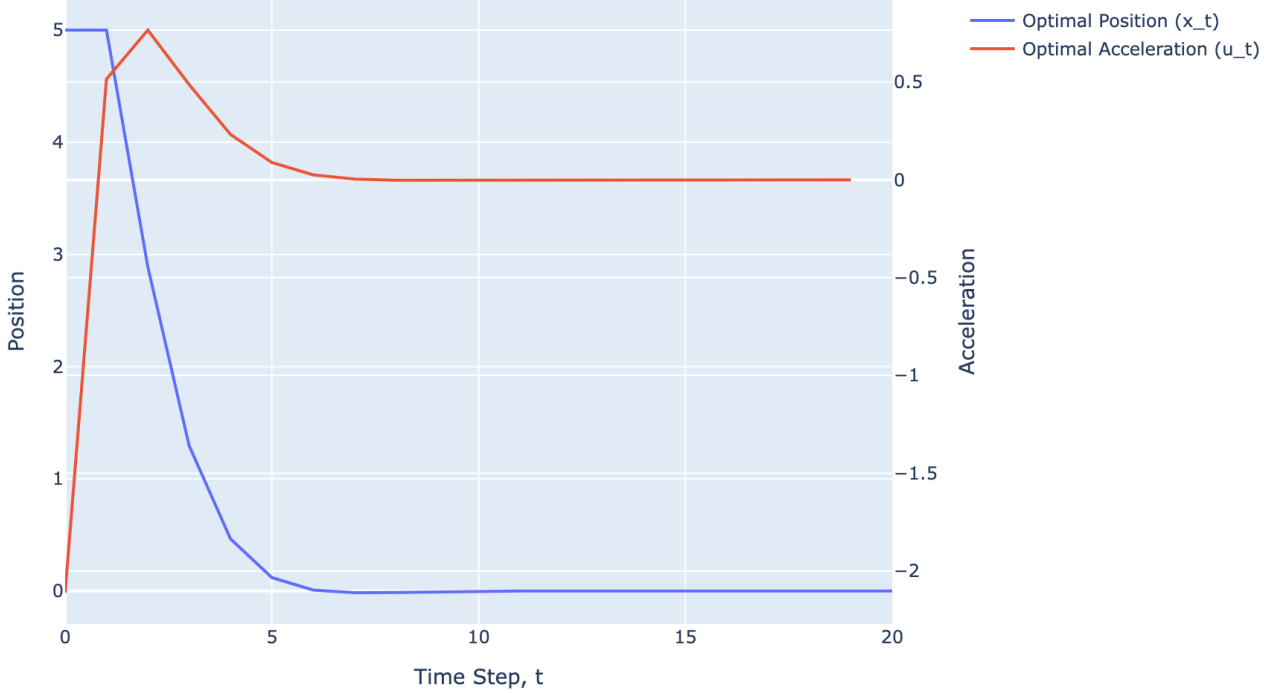
```python
def plot_optimal_simulation(
    T: int,
    A: np.ndarray,
    B: np.ndarray,
    Q: np.ndarray,
    Q_f: np.ndarray,
    r: float,
    x_0: np.ndarray,
    dynamics: Callable,
    title: str | None = None,
) -> tuple[list[np.ndarray | None], list[np.ndarray | None]]:
    x_optimal, u_optimal = simulate_optimal(T, A, B, Q, Q_f, r, x_0, dynamics)
    p_optimal = [x[0][0] for x in x_optimal]
    v_optimal = [x[1][0] for x in x_optimal]

    title = f"[Optimal Simulation] Q = {Q}, r = {r}, Q_f = {Q_f}" if title is None else title

    # To match the time steps, x is one longer
    # u_optimal.append(np.ndarray([0]))

    t = list(range(len(p_optimal)))
    fig = make_subplots(specs=[[{"secondary_y": True}]])
    # Optimal Position Trace
    fig.add_trace(
        go.Scatter(
            name="Optimal Position (x_t)",
            x=t,
            y=p_optimal,
        )
    )
    # Optimal Acceleration Trace
    u_optimal = [u[0][0] for u in u_optimal]
    fig.add_trace(go.Scatter(x=t, y=u_optimal, name="Optimal Acceleration (u_t)"), secondary_y=True)
    fig.update_layout(title=title, xaxis_title="Time Step, t", yaxis_title="Position")
    fig.update_yaxes(title_text="Acceleration", secondary_y=True)

    return fig


for penalty_name, param_set in penalty.items():
    for Q, r, Q_f in param_set:
        fig = plot_optimal_simulation(T, A, B, Q, Q_f, r, x_0, dynamics)
        fig.show()
```

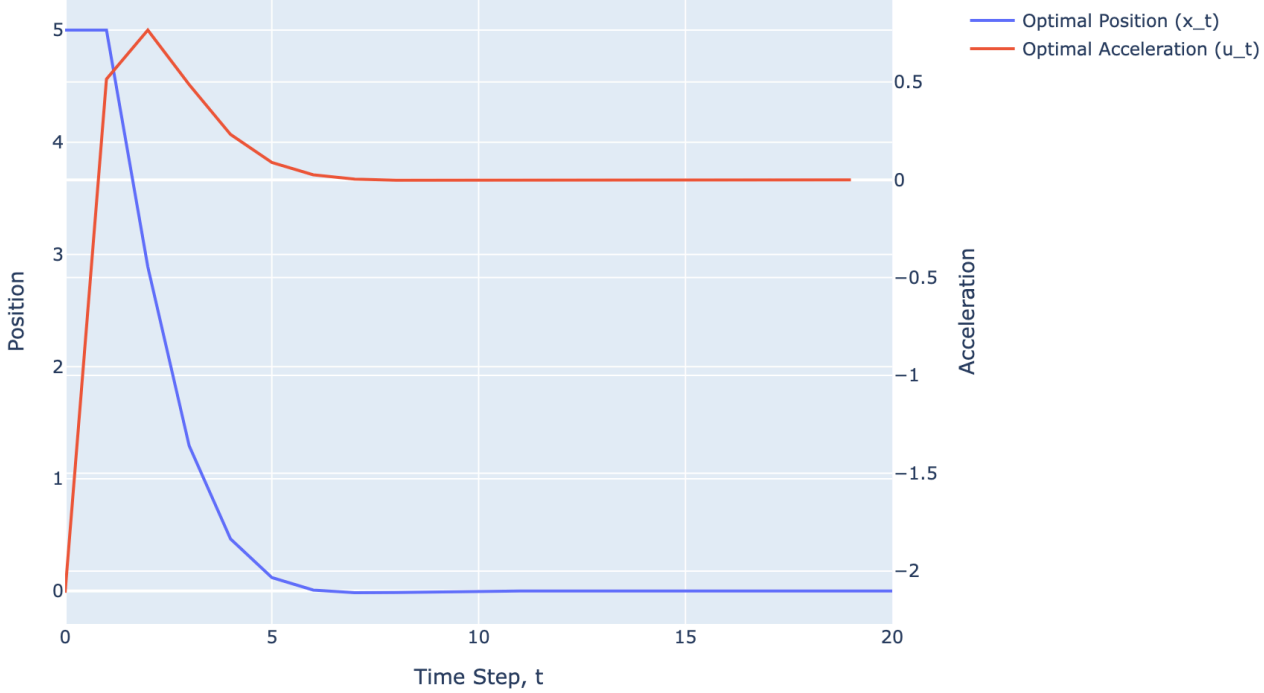

[Optimal Simulation] Q = [[0.1 0.] [0. 1.]], r = 1, Q_f = [[1. 0.] [0. 1.]]
Optimal Position (x_t)
Optimal Acceleration (u_t)
Position
Acceleration
Time Step, t
0
1
2
3
4
5
0.2
0
−0.2
−0.4
−0.6
−0.8
0
5
10
15
20

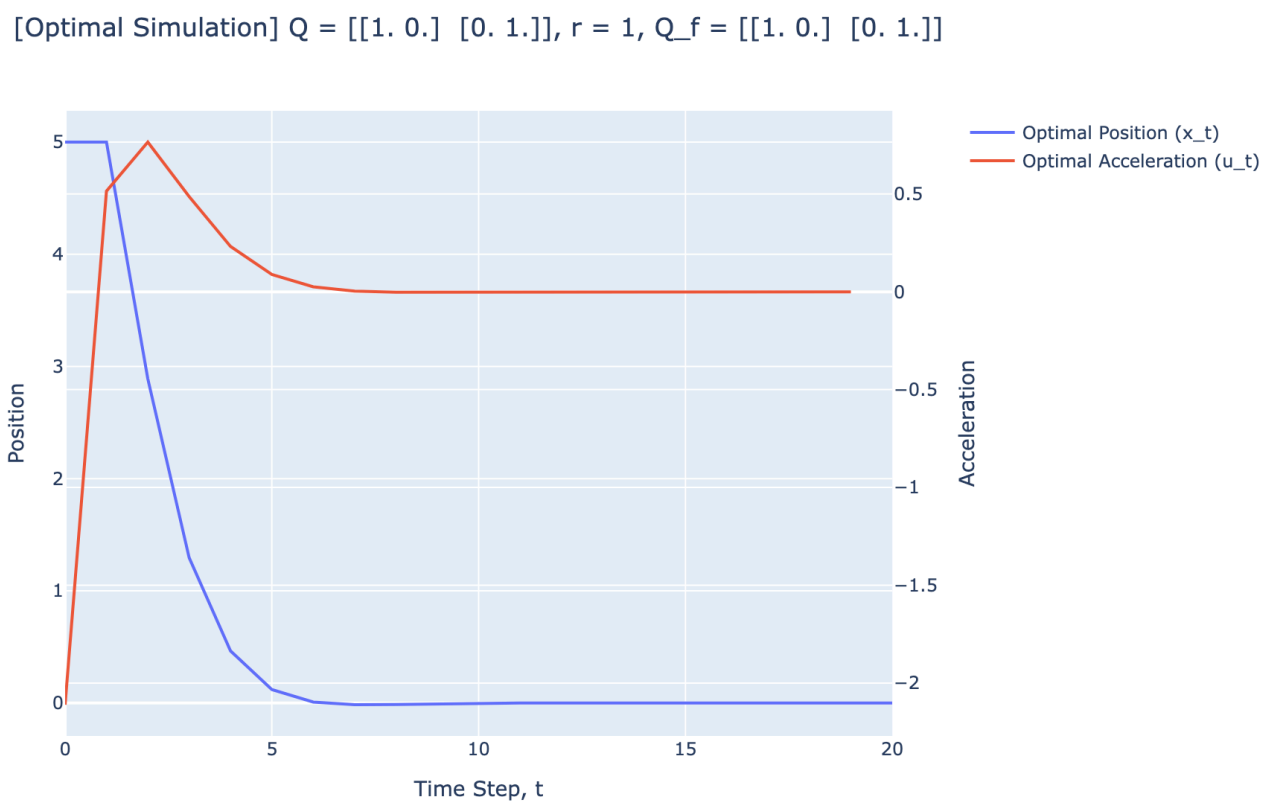

[Optimal Simulation] Q = [[10 0] [0 1]], r = 1, Q_f = [[1. 0.] [0. 1.]]
Optimal Position (x_t)
Optimal Acceleration (u_t)
Position
Acceleration
Time Step, t
0
1
2
3
4
5
−4
−3
−2
−1
0
1
2
3
0
5
10
15
20


[Optimal Simulation] Q = [[1. 0.] [0. 1.]], r = 0.1, Q_f = [[1. 0.] [0. 1.]]
Optimal Position (x_t)
Optimal Acceleration (u_t)
Position
Acceleration
Time Step, t
0
1
2
3
4
5
0
5
10
15
20
1.5
1
0.5
0
−0.5
−1
−1.5
−2
−2.5
−3


[Optimal Simulation] Q = [[1. 0.] [0. 1.]], r = 10, Q_f = [[1. 0.] [0. 1.]]
Optimal Position (x_t)
Optimal Acceleration (u_t)
Position
Acceleration
Time Step, t
0
1
2
3
4
5
0
5
10
15
20
0.2
0
−0.2
−0.4
−0.6
−0.8
−1


[Optimal Simulation] Q = [[1. 0.] [0. 1.]], r = 1, Q_f = [[3. 0.] [0. 3.]]
Optimal Position (x_t)
Optimal Acceleration (u_t)
Position
Acceleration
Time Step, t
0
1
2
3
4
5
0
5
10
15
20
0.5
0
−0.5
−1
−1.5
−2


[Optimal Simulation] Q = [[1. 0.] [0. 1.]], r = 1, Q_f = [[10. 0.] [0. 10.]]
Optimal Position (x_t)
Optimal Acceleration (u_t)
Position
Acceleration
Time Step, t
0
1
2
3
4
5
0.5
0
−0.5
−1
−1.5
−2
0
5
10
15
20


[Optimal Simulation] Q = [[1. 0.] [0. 1.]], r = 1, Q_f = [[1. 0.] [0. 1.]]
Optimal Position (x_t)
Optimal Acceleration (u_t)
Position
Acceleration
Time Step, t
0
1
2
3
4
5
0
5
10
15
20
0.5
0
−0.5
−1
−1.5
−2